BudgetFlow: Value-Aware Budget Governance for Agent Tasks

Anonymous submission

Abstract

Multi-step AI agent tasks consume variable amounts of model budget, and tasks in a batch often have unequal value. We study *Batch-Level Task Budget Allocation*: a task batch shares one hard budget, and scarce model opportunities should flow toward the tasks that create the most verified value.

We introduce BudgetFlow, a value-aware budget governance framework for task batches. Its general interface combines task-level signals—Task Value, Estimated Token Cost, Model Fit, and a Verifier—with batch-level Shared Budget state. BudgetFlow uses the first three task signals and the Shared Budget to allocate budget and model tiers, while the Verifier settles outcomes. This separates allocation signals from outcome settlement, so the same interface can describe coding, support, document, spreadsheet, and research task batches once each domain supplies a trusted verifier.

We evaluate BudgetFlow on SWE-bench-style verified bug-fixing tasks under fixed task sets and shared hard budgets. The paper reports Resolved Rate, Total Resolved Value (TRV), and TRV per Dollar. Across one 4-policy run and three 5-policy runs over 30-task batches, BudgetFlow produces two value-aware frontier results: with metadata-only estimates, it resolves higher value than the strong-model-only baseline at about 16% lower spend; with prior-batch observations, it leads all three reported metrics. The other main batches identify Strong Coverage cases, where broad strong-model use is already cost-effective under the cap. To make this evidence interpretable, we propose a frontier-mode framework with three modes: Strong Coverage, Model Fit, and Value-Aware Budget Flow. The third mode is the central target of BudgetFlow.

Introduction

AI agents execute multi-step tasks with uneven model spend, and tasks in a batch rarely have equal value. We study *Batch-Level Task Budget Allocation*: many tasks share one hard budget, and scarce model opportunities should flow toward tasks that create the most verified value.

This setting admits several frontier modes. Strong Coverage: broad strong-model use is cost-effective. Model Fit: task-to-model matching dominates. BudgetFlow targets Value-Aware Budget Flow: when tasks differ in value, estimated token cost, and model-tier upside, the policy allocates scarce opportunities across the full batch.

BudgetFlow is a budget governance framework for this setting. It uses four task-level signals and one batch-level budget state:

- **Task Value**: the pre-registered value gained if the task is resolved;
- **Estimated Token Cost**: the pre-run estimate of token cost used for cap planning;
- **Model Fit**: evidence about how well each model tier fits the task;
- **Verifier**: the outcome rule that maps a completed attempt to resolved or unresolved;
- **Shared Budget**: the hard batch-level resource constraint shared by all tasks.

The first three signals and Shared Budget drive allocation; the Verifier settles outcomes. We instantiate the interface on SWE-bench-style verified coding tasks and report Resolved Rate, TRV, and TRV per Dollar.

This paper makes three contributions:

1. We formulate Batch-Level Task Budget Allocation as a value-governance problem with pre-registered Task Value, Estimated Token Cost, Model Fit, a shared hard budget, and a Verifier-settled objective.
2. We present a generalizable BudgetFlow framework and interface that allocate model opportunities through Task Value, Estimated Token Cost, Model Fit, Shared Budget, and a Verifier-settled outcome contract across task domains.
3. We propose a frontier-mode framework for interpreting shared-budget agent batches, identifying **Strong Coverage**, **Model Fit**, and **Value-Aware Budget Flow** as three frontier modes. **Value-Aware Budget Flow** is the central mode targeted by BudgetFlow.

Figure 1 summarizes the BudgetFlow loop. Adjacent systems decide which model answers one request or how one task spends its own budget. BudgetFlow instead repeatedly allocates scarce budget and strong-model opportunities across a task pool under one shared hard budget, then settles outcomes with a Verifier and updates remaining budget and verified TRV.

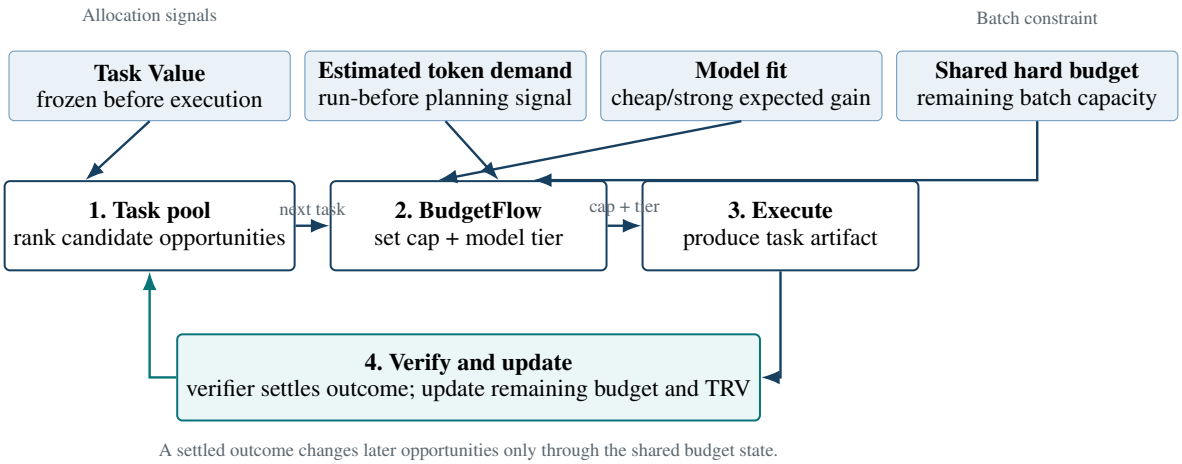

Figure 1: BudgetFlow overview. Four auditable inputs determine a task opportunity. BudgetFlow allocates a cap and model tier across the task pool, then the Verifier settles the artifact and updates the remaining shared hard budget and verified TRV for later tasks.

Related Work

We organize related work by the decision problem it solves. Conference venues are not a ranking signal: newer papers are placed only by what decision they make.

Per-request cost–quality routing. RouteLLM learns per-query routers from preference data (Ong et al. 2025). Cascade Routing studies model routing and cascading for individual queries (Dekoninck, Baader, and Vechev 2025). RouteNLP targets enterprise query routing under quality constraints (Guo, Wu, and Yiu 2026). UCCI calibrates uncertainty for cost-optimal cascade routing (Kotte 2026). ZeroRouter maps requests and models into a shared latent space for multi-objective routing and low-cost onboarding of new models (Yan et al. 2026). Capability Instruction Tuning / MODEL-SAT selects models from capability instructions (Zhang, Zhan, and Ye 2025). ICL-Router and CP-Router further study request-level routing with in-context model representations or uncertainty-aware LLM/LRM choice (Wang et al. 2026; Su et al. 2026). These methods make model selection a first-class cost–quality problem for one prompt or one request. They do not allocate one shared hard budget across many tasks with unequal value.

Within-task budgeted agent control. INTENT plans costly tool use inside a single agent task under that task's own budget (Liu et al. 2026). BAMAS builds a multi-agent system for one task under a per-task cost cap by choosing models and collaboration structure (Yang et al. 2026). Online Multi-LLM Selection adapts model choice over multi-turn interaction with a per-dialogue budget when the next prompt evolves as a black box (Poon et al. 2026). CCPO orchestrates cheaper and stronger models inside one question under a reliability constraint (Si et al. 2026). STEER switches between small and large models at reasoning steps using confidence signals (Lee et al. 2026). These methods optimize spend or model switches inside one task or one interaction. Even when a budget exists, it belongs to that task or dialogue, not to a batch of competing tasks.

Cost-aware agent runtimes. OpenSquilla is a token-efficient agent runtime with model routing, diagnostics, replay, and cost-per-success framing (OpenSquilla contributors 2026). It strengthens the requirement that agent cost and execution behavior be measurable. Closely related harness-evaluation work such as Claw-SWE-Bench further stresses fair harness comparison and cost reporting (Zheng et al. 2026), but it evaluates adapters rather than cross-task budget governance. BudgetFlow uses a different decision unit: many tasks share one hard budget, and scarce strong-model opportunities should flow toward tasks that create the most verified value under frozen Task Value.

Problem Formulation

Let a task batch contain tasks $\mathcal{T} = \{1, \ldots, n\}$ and let B denote the Shared Budget. Each task i has a pre-registered value $v_i > 0$, an estimated token cost $d_i > 0$, and model-fit evidence $m_{i,k}$ for each model tier k. A policy π chooses a sequence of actions that spend budget on model calls and task attempts. Let $c_i(\pi)$ be the spend on task i, and let

$$\sum_{i=1}^{n} c_i(\pi) \leq B$$

be the shared hard-budget constraint.

The budget is shared at the batch level. A policy can spend more on one task only by leaving less budget for later tasks in the same batch. This coupling is what separates the problem from independent per-task routing: a locally reasonable strong-model call can be globally poor if it consumes budget needed by a higher-value task.

Each task has a Verifier V_i that maps the final task evidence to a binary outcome:

$$r_i(\pi) = V_i(\pi, i) \in \{0, 1\}.$$

The primary value objective is Total Resolved Value:

$$\mathrm{TRV}(\pi) = \sum_{i=1}^{n} v_i \, r_i(\pi).$$

We report three headline metrics:

$$\mathrm{ResolvedRate}(\pi) = \frac{1}{n} \sum_{i=1}^{n} r_i(\pi),$$

$$\mathrm{TRVperDollar}(\pi) = \frac{\mathrm{TRV}(\pi)}{\sum_i c_i(\pi)}.$$

The Verifier settles outcomes after execution; allocation uses Task Value, Estimated Token Cost, Model Fit, and Shared Budget state. In SWE-bench, V_i is a test-based harness. Task Value is fixed before execution; Estimated Token Cost is a pre-run planning estimate; Model Fit can come from catalog priors or prior outcomes.

BudgetFlow

BudgetFlow maintains a Shared Budget ledger across the task batch. For each task, it estimates whether the next model opportunity should use the cheap model, the strong model, or defer additional spend. The ledger is hard: Shared Budget gates each planned action. The goal is to turn the general interface in Section into a concrete value-aware allocation policy.

The policy uses three allocation inputs:

- v_i, the Task Value;
- d_i, the Estimated Token Cost;
- $m_{i,k}$, the Model Fit for tier k.

These inputs enter distinct decisions. Task Value prioritizes tasks that create more value if verified. Estimated Token Cost becomes cap planning: the policy estimates how much token cost a task may need before it can produce a useful artifact. Model Fit estimates whether the strong model is likely to improve the outcome relative to the cheap model. Shared Budget controls affordability and stop/defer decisions.

Let h denote budget pressure, with higher h indicating tighter Shared Budget state. Let k_c be the cheap model and k_s be the strong model. A simplified task-start score for giving task i a strong-model opportunity is

$$S_i = \frac{v_i \cdot \max(m_{i,k_s} - m_{i,k_c}, 0)}{\max(d_i, \epsilon)} \cdot q(h),$$

where d_i is the task's Estimated Token Cost and $q(h)$ reduces strong-model access as Shared Budget tightens. At task start, BudgetFlow assigns a route and cap; during execution it can grant a strong-model opportunity or stop/defer when upside is low or the batch budget is threatened. Allocation and outcome settlement stay separate: the Verifier evaluates the final artifact after the attempt.

Algorithm 1 BudgetFlow batch policy sketch

Initialize Shared Budget state $R \leftarrow B$.
for task i in the batch **do**
  Read $(v_i, d_i, m_{i,k_c}, m_{i,k_s})$ and current R.
  Set an initial task cap from Task Value, Estimated Token Cost, and R.
  Choose cheap or strong model from Model Fit, value upside, and affordability.
  Execute the task while updating spend and Shared Budget.
  Continue, **stop**, or **defer** using value upside and budget pressure.
  Settle the final artifact with V_i and record r_i.
end for
Report Resolved Rate, TRV, TRV per Dollar, and total spend.

Policy	Task Value	Model choice
cheap-model-only		fixed cheap
strong-model-only		fixed strong
learned task-router		value-blind learned
budget-only		budget pressure
BudgetFlow	yes	value-aware

Table 1: Policy inputs under the same shared hard budget and verifier.

Experimental Setup

Task set. We evaluate on fixed 30-task SWE-bench-style bug-fixing batches. Each task requires an agent to inspect a repository, edit code, and produce a patch. Outcomes are scored by a local test-based harness, and every run exports official-format predictions for independent re-evaluation. Task values are pre-registered before execution.

Policies. Table 1 compares the five policies by the inputs they use. All policies run under the same shared hard budget and the same verifier. The key contrast is whether Task Value enters allocation of scarce model opportunities.

All policies share the same task order, budget protocol, model-tier catalog, and verifier. Batches differ in whether Estimated Token Cost/Model Fit use metadata-only estimates or prior-batch observations, and in the value ledger; hard caps are \$9.95 (5-policy) or \$10.44 stage-prefix pressure (4-policy).

Metrics and sensitivity. We report Resolved Rate, TRV, and TRV per Dollar. Sensitivity replays one completed 5-policy batch under alternate value profiles or budget caps without new paid runs.

Results

Batch reading	Policy	Resolved Rate	Spend	TRV	TRV per Dollar
BudgetFlow Pareto: higher TRV at lower spend	BudgetFlow	14/30 (46.7%)	$7.98	**18.50**	**2.32**
	strong-model-only	**15/30 (50.0%)**	$9.46	18.00	1.90
Strong Coverage: larger strong-model lead	strong-model-only	**17/30 (56.7%)**	$9.95	**20.00**	**2.01**
	BudgetFlow	15/30 (50.0%)	$9.95	17.50	1.76
Strong Coverage: close strong-model lead	strong-model-only	**18/30 (60.0%)**	$9.88	**19.00**	**1.92**
	BudgetFlow	17/30 (56.7%)	$9.95	18.50	1.86
BudgetFlow leads all: all primary metrics	BudgetFlow	**16/30 (53.3%)**	$9.95	**18.00**	**1.81**
	learned task-router	15/30 (50.0%)	$9.95	17.00	1.71
	strong-model-only	15/30 (50.0%)	$9.95	16.50	1.66

Table 2: Main paid evidence. Each group is one 30-task batch. Red bold: BudgetFlow frontier signals; blue bold: strongest control.

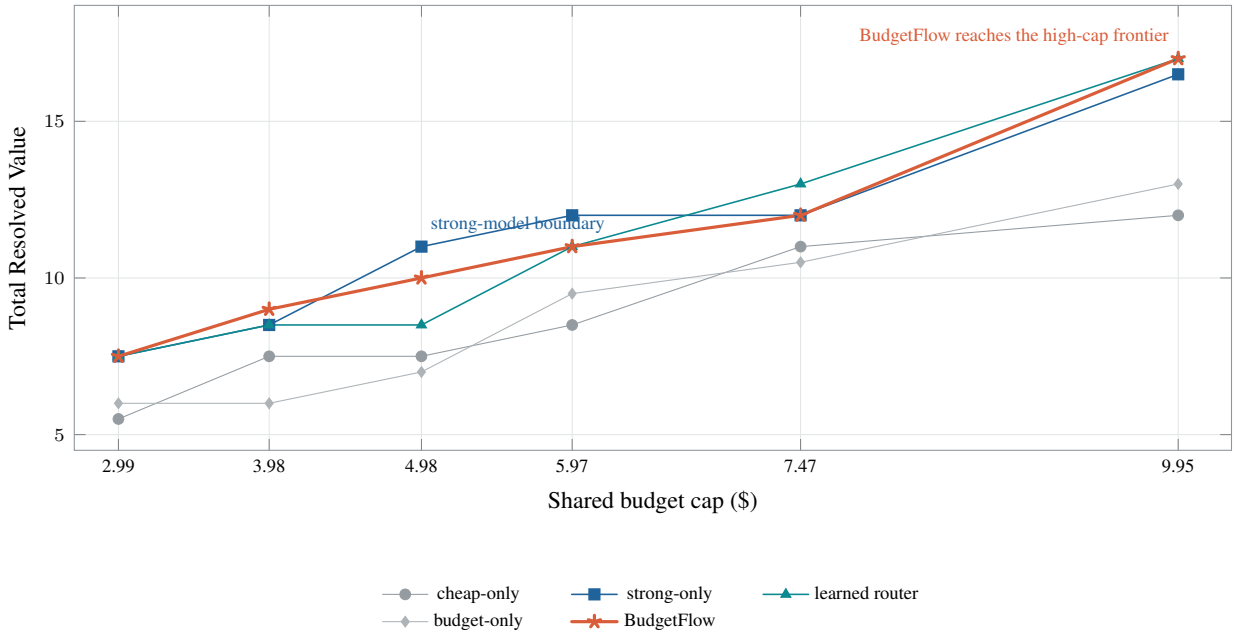

Figure 2: Cost-value curve from budget sensitivity on the completed 5-policy batch. Outcomes and task order are fixed; each point replays accounting at a shared budget cap. The curve shows the operating conditions in which broad strong-model coverage or value-aware batch allocation forms the observed frontier.

Reading the evidence. The 4-policy BudgetFlow Pareto case reaches **TRV 18.50 at $7.98**, compared with TRV 18.00 at $9.46 for strong-model-only: higher verified value at about 16% lower spend despite one fewer resolved task. In the 5-policy BudgetFlow leads-all case, BudgetFlow leads Resolved Rate, TRV, and TRV per Dollar. The Strong Coverage cases are equally important boundary evidence: broad strong-model use is the observed frontier when it converts the shared hard budget to verified value efficiently enough that selective allocation does not improve it.

Interpretation. The curve is a no-paid replay of the completed 5-policy batch: task order and verifier outcomes remain fixed while accounting is replayed at sampled shared budget caps. It shows where Value-Aware Budget Flow reaches the observed frontier and where a control reaches or shares it. The evidence therefore does not claim that BudgetFlow always beats the strongest configured tier; it identifies the operating conditions in which shared-budget value allocation improves the cost-value frontier.

Sensitivity Analysis

Sensitivity replays fixed outcomes from the 5-policy BudgetFlow leads-all case. Figure 2 is the main cost-value curve: budget replay moves the observed frontier between Strong Coverage and Value-Aware Budget Flow. Re-scoring equal, criticality, compressed, and expanded Task Value profiles leaves BudgetFlow ahead of the best control by +1.00 TRV in every profile (median +1.00 over 64 permutations).

Discussion and Conclusion

The SWE-bench experiments instantiate a domain-neutral contract: tasks expose Task Value, Estimated Token Cost, Model Fit, and a Verifier under one Shared Budget. Frontier-mode reporting makes mixed outcomes interpretable—Strong Coverage when the strong model is broadly cost-effective, Model Fit when tier routing dominates under serving discounts, and Value-Aware Budget Flow when unequal task value forces cross-task tradeoffs. Future work can add finer within-task control, auditable learning from completed batches, and serving-aware multi-tenant scheduling above inference substrates.

BudgetFlow formulates Batch-Level Task Budget Allocation and separates allocation signals from Verifier-settled outcomes. On SWE-bench, it improves the spend–TRV frontier in one batch and leads Resolved Rate, TRV, and TRV per Dollar in another under the same hard cap. The transferable object is the interface for governing verified value under fixed resource constraints.

References

Dekoninck, J.; Baader, M.; and Vechev, M. 2025. A Unified Approach to Routing and Cascading for LLMs. In *International Conference on Machine Learning*.

Guo, D.; Wu, J.; and Yiu, S. M. 2026. RouteNLP: Closed-Loop LLM Routing with Conformal Cascading and Distillation Co-Optimization. In *ACL Industry Track*.

Kotte, V. 2026. UCCI: Calibrated Uncertainty for Cost-Optimal LLM Cascade Routing. *arXiv preprint arXiv:2605.18796*.

Lee, S.; Kim, D.; Koh, H.; Yang, N.; and Jung, K. 2026. Confidence-Guided Stepwise Model Routing for Cost-Efficient Reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Liu, H.; Tian, C.; An, N.; Wang, Z.; Lu, P.; Yu, C.; and Qi, Q. 2026. Budget-Constrained Agentic Large Language Models: Intention-Based Planning for Costly Tool Use. *arXiv preprint arXiv:2602.11541*.

Ong, I.; Almahairi, A.; Wu, V.; Chiang, W.-L.; Wu, T.; Gonzalez, J. E.; Kadous, M. W.; and Stoica, I. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *International Conference on Learning Representations*.

OpenSquilla contributors. 2026. OpenSquilla: Token-Efficient AI Agent. https://github.com/opensquilla/opensquilla. Accessed 2026-07-06.

Poon, M.; Dai, X.; Liu, X.; Kong, F.; Lui, J. C. S.; and Zuo, J. 2026. Online Multi-LLM Selection via Contextual Bandits Under Unstructured Context Evolution. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Si, W.; Jang, S.; Lee, I.; and Bastani, O. 2026. Conformal Constrained Policy Optimization for Cost-Effective LLM Agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Su, J.; Lin, F.; Feng, Z.; Zheng, H.; Wang, T.; Xiao, Z.; Zhao, X.; Liu, Z.; Cheng, L.; and Wang, H. 2026. CP-Router: An Uncertainty-Aware Router Between LLM and LRM. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Wang, C.; Li, H.; Zhang, Y.; Chen, L.; Chen, J.; Jian, P.; Zhang, Q.; and Hu, S. 2026. ICL-Router: In-Context Learned Model Representations for LLM Routing. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Yan, C.; Zhang, W.; Ning, Z.; Xu, F.; Tao, Z.; Zhang, L.; Yin, B.; and Zhang, Y. 2026. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Yang, L.; Luo, J.; Liu, X.; Lou, Y.; and Chen, Z. 2026. BAMAS: Structuring Budget-Aware Multi-Agent Systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Zhang, Y.-K.; Zhan, D.-C.; and Ye, H.-J. 2025. Capability Instruction Tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Zheng, M.; Han, K.; Li, B.; Xu, H.; Tian, Y.; He, W.; Zhou, H.; Guo, J.; Hu, H.; Ma, L.; Xu, C.; Dai, G.; Xia, L.; Wei, Y.; Wang, Y.; and Wang, Y. 2026. Claw-SWE-Bench: A Benchmark for Evaluating OpenClaw-style Agent Harnesses on Coding Tasks. *arXiv preprint arXiv:2606.12344*.